

TCC70xx MCU BSP

Getting Started

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	4
2	Terms and Abbreviations	5
3	Board Description	6
3.1	EVB Description	6
3.2	Boot Mode	8
3.2.1	TCC7022_25_35_45_MCU EVB	8
4	Download MCU BSP	9
5	Build Guide.....	10
5.1	GCC Environment	10
5.1.1	Install Toolchain.....	10
5.1.2	How to Build MCU BSP using GNU Make Environment	11
5.2	Green Hills IDE Environment	12
5.2.1	Green Hills Version.....	12
5.2.2	Make New Project Using Green Hills IDE	14
5.2.3	Build TCC70xx MCU BSP.....	23
5.3	Preparation Image for Normal Booting.....	25
6	Firmware Downloader (FWDN)	26
6.1	System Requirement.....	26
6.1.1	Software Requirement.....	26
6.1.2	Hardware Requirement	26
6.2	Install VTC Driver	27
6.3	FWDN Sequence	27
7	How to debug by using JTAG Debugger.....	29
8	References	30
9	Revision History	31
	Rev. 0.10: 2021-09-03	31

Figures

Figure 3.1	TCC7022_25_35_45_MCU EVB	6
Figure 3.2	Boot Mode Switch.....	8
Figure 3.3	UART Boot Mode	8
Figure 3.4	Normal Boot Mode.....	8
Figure 5.1	Output Files for GCC Environment	11
Figure 5.2	MULTI Project Manager	12
Figure 5.3	Write Project Name	14
Figure 5.4	Select Operating System	14
Figure 5.5	Select Processor Family	14
Figure 5.6	Select Target Board	15
Figure 5.7	Generated Project Files	15
Figure 5.8	Generate New Program	16
Figure 5.9	Select Source Code Directory	16
Figure 5.10	Select Program Layout	16
Figure 5.11	Completion of Initial Project Setup	17
Figure 5.12	Add New Source Code Item.....	17
Figure 5.13	Add Source File	18
Figure 5.14	Complete Adding Source File	18
Figure 5.15	Add Assembly File (.asm)	19
Figure 5.16	Set up Build Option.....	19
Figure 5.17	Configure Include Path.....	20
Figure 5.18	Add Include Path	20
Figure 5.19	Add Include Path Manually	21
Figure 5.20	Set up Memory Layout	21
Figure 5.21	Output File Name	22
Figure 5.22	Open Project File	23
Figure 5.23	Modify EVB Definition.....	23
Figure 5.24	Build Project.....	24
Figure 5.25	Build Completion.....	24
Figure 6.1	VTC Driver Install Window.....	27
Figure 6.2	Input COM port number	27
Figure 6.3	FWDN Process.....	28
Figure 6.4	Initial Log of MCU F/W.....	28

Figure 7.1 Trace32 Script for Debug	29
---	----

Tables

Table 2.1 Terms and Abbreviation	5
Table 5.1 Green Hills Compiler Version	13
Table 6.1 Software Requirement.....	26
Table 6.2 Hardware Requirement.....	26

1 INTRODUCTION

This document provides a usage of TCC70xx MCU BSP as follows:

- Board description (TCC7022_25_35_45_MCU Evaluation Board)
- Download MCU BSP
- Build guide for GCC and Green Hills Multi IDE
- How to make pflash image
- FWDN guide for downloading image
- How to debug by using JTAG debugger

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

EVb	Evaluation Board
F/W	Firmware
FWDN	Firmware Downloader PC tool by Telechips
GCC	GNU Compiler Collection
Green Hills IDE	Integrated Development Environment (IDE) by Green Hills software
BU	Battery Unit
JTAG	Joint Test Action Group
T32 (Trace32)	One of JTAG debugger
pflash	Program Flash memory

3 BOARD DESCRIPTION

This chapter describes TCC7022_25_35_45_MCU EVB (hereinafter referred to as EVB). TCC70xx MCU BSP can be executed on TCC7022_25_35_45_MCU EVB.

3.1 EVB Description

The figure below shows TCC7022_25_35_45_MCU EVB. TCC70xx MCU BSP was tested under TCC7022_25_35_45_MCU EVB V0.2.1.

- **FWDN USB-C Port** : USB Type C port to connect with FWDN PC Tool
- **UART USB-C Port** : UART debug port for MCU UART log
- **DC 12V** : Main Power
- **ILL Switch** : Switch for ILL test
- **ACC Switch** : Switch for ACC test
- **Power On Switch** : Switch for power on reset
- **SYS Reset Switch** : Switch for reset of system
- **Boot Mode Switch** : Switch for selecting the boot mode of system
- **JTAG Header** : Header for JTAG
- **External Key Header** : Header for key sub-board
- **ILL Potentiometer** : Potentiometer for ILL test
- **Rotary Key** : Encoder Switch for Key test
- **ADC Potentiometer** : Potentiometer for ADC test
- **CAN and LIN Output Header** : Header for selecting output channel for CAN and LIN
- **Connector for CAN and LIN** : Connector for CAN and LIN

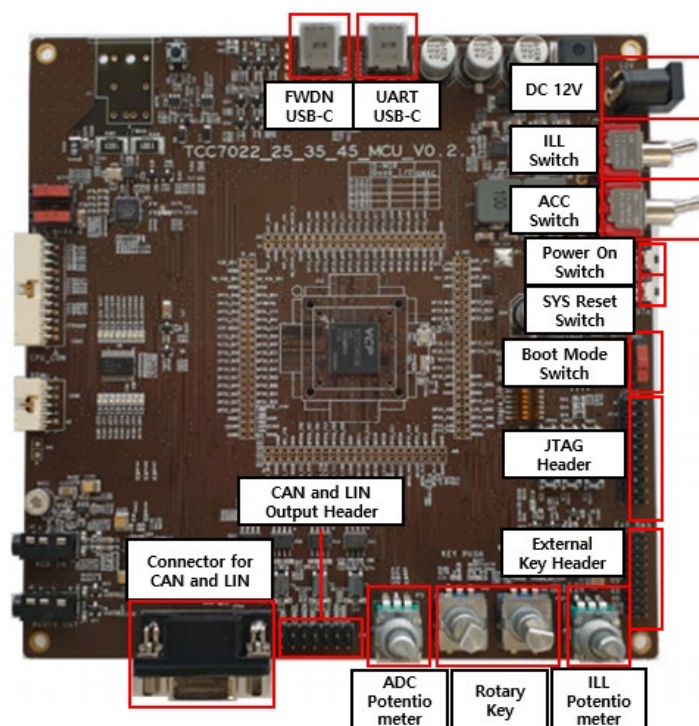


Figure 3.1 TCC7022_25_35_45_MCU EVB

For more detailed information on TCC7022_25_35_45_MCU EVB usage, refer to “*TCC70xx Common Hardware-Quick Start Guide for TCC7022_25_35_45_MCU EVB*” [2] or “*TCC70xx Common Hardware-User Guide for TCC7022_25_35_45_MCU EVB*” [3].

3.2 Boot Mode

The TCC7022_25_35_45_MCU EVB has boot mode switch for booting configuration. The TCC70xx MCU BSP requires Normal boot mode and UART boot mode for executing F/W and FWDN. For more detailed information on TCC7022_25_35_45_MCU EVB boot mode, refer to [2] and [3].

3.2.1 TCC7022_25_35_45_MCU EVB



Figure 3.2 Boot Mode Switch

Pin	Value	Figure	
BM(SW20)	High(1)		

Figure 3.3 UART Boot Mode

Pin	Value	Figure	
BM(SW20)	Low(0)		

Figure 3.4 Normal Boot Mode

4 DOWNLOAD MCU BSP

This chapter describes how to download the MCU BSP package. You can get the MCU BSP package as follows:

- Prepare git environment.
- Clone repository using “clone” command as follows: `git clone ssh://git.telechips.com/vcp/tcc70xx_mcu_bsp.git`
Note: Above git is available from MCU BSP v0.1.0.
- Access permission is required to clone the release git repository. If you have permission problem, contact Telechips. [4]
Check all released branch using “branch” command as follows: `git branch -a`
- Move to release branch using “checkout” command as follows: `git checkout branch name`
- For more detailed information on git, refer to git manual. You can get it by using “git help” command.

5 BUILD GUIDE

There are two build options as described below.

- GCC environment
 - This is default build environment.
 - Free license
- Green Hills IDE environment
 - Windows dedicated build environment.
 - No Free license

5.1 GCC Environment

Important: The normal operation of MCU BSP is guaranteed only in Green Hills IDE Environment.

5.1.1 Install Toolchain

Basically, MCU BSP uses “*gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi*”. Refer to the Makefile.

- Toolchain for GCC Environment

vcp_mcu/sources/build/TCC70xx/gcc/Makefile

```
MCU_BSP_TOOLCHAIN_PATH      ?= /opt/gcc-linaro-7.2.1-2017.11-x86_64_arm-eabi
MCU_BSP_TOOLCHAIN_PREFIX    ?= $(MCU_BSP_TOOLCHAIN_PATH)/bin/arm-eabi-

LIBCDIR      := $(MCU_BSP_TOOLCHAIN_PATH)/arm-eabi/libc/usr/lib
LIBGCCDIR    := $(MCU_BSP_TOOLCHAIN_PATH)/lib/gcc/arm-eabi/7.2.1
LIBGCC       := -L$(LIBCDIR) -L$(LIBGCCDIR) -lc -lm -lgcc
AS           := $(MCU_BSP_TOOLCHAIN_PREFIX)as
CC           := $(MCU_BSP_TOOLCHAIN_PREFIX)gcc
LD           := $(MCU_BSP_TOOLCHAIN_PREFIX)ld
SIZE         := $(MCU_BSP_TOOLCHAIN_PREFIX)size
OBJCOPY      := $(MCU_BSP_TOOLCHAIN_PREFIX)objcopy
OBJDUMP      := $(MCU_BSP_TOOLCHAIN_PREFIX)objdump
INCLUDES     += -I$(MCU_BSP_BUILD_ROOT_PATH)
INCLUDES     += -Iinclude
INCLUDES     += -I$(LIBCDIR)/../include
INCLUDES     += -I$(LIBGCCDIR)/include
```

5.1.2 How to Build MCU BSP using GNU Make Environment

1. Go to "`vcp_mcu/sources/build/tcc70xx/gcc`" path
2. Edit Makefile if you need to change build options. Refer to the Build Flags in Makefile.

■ Build Option for GCC Environment

`vcp_mcu/sources/build/TCC70xx/gcc/Makefile`

```
# Build Flags
MCU_BSP_BUILD_FLAGS_RAM_256           ?= 0
MCU_BSP_BUILD_FLAGS_RAM_512           ?= 1

MCU_BSP_BUILD_FLAGS_VERBOSE            ?= 0
MCU_BSP_BUILD_FLAGS_DEBUG              ?= 1

MCU_BSP_BUILD_FLAGS_TARGET_OS_FREERTOS ?= 1
MCU_BSP_BUILD_FLAGS_TARGET_OS_UCOS     ?= 0

MCU_BSP_BUILD_FLAGS_OS_CPU_VFP_NONE    ?= 1
MCU_BSP_BUILD_FLAGS_OS_CPU_VFP_D16     ?= 0
MCU_BSP_BUILD_FLAGS_PFLASH_BOOT        ?= 1
```

3. Execute "make clean" to remove all object files.
4. Execute "make" to build.
5. You can see the MCU BSP output files as follows:

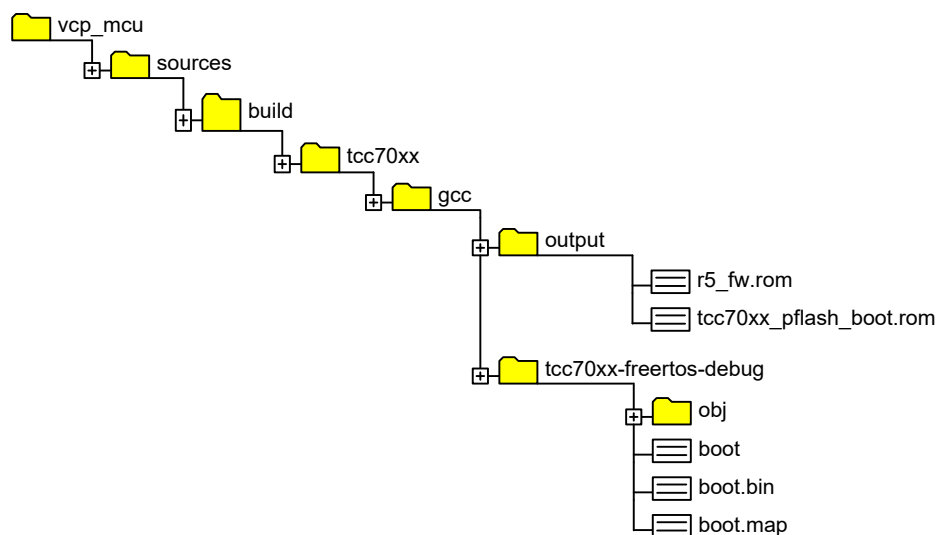


Figure 5.1 Output Files for GCC Environment

MCU BSP output files are located in "tcc70xx-freertos-release" or "tcc70xx-freertos-debug" and "output" directory. It depends on the build option (**MCU_BSP_BUILD_FLAGS_DEBUG**) in Makefile.

- A. tcc70xx-freertos-xxx/obj : object files
- B. tcc70xx-freertos-xxx/boot : MCU BSP elf file
- C. tcc70xx-freertos-xxx/boot.bin : MCU BSP binary file
- D. tcc70xx-freertos-xxx/boot.map : MCU BSP map file
- E. output/r5_fw.rom: Final MCU BSP rom file. It is created by adding a header to the binary file ("boot.bin"). It can be updated to the target EVB after being included in pflash image("tcc70xx_pflash_boot.rom"). Refer to Chapter 5.3 Preparation Image for Normal Booting.

5.2 Green Hills IDE Environment

Important: MCU BSP is guaranteed only in Green Hills IDE Environment.

5.2.1 Green Hills Version

5.2.1.1 Green Hills IDE Version

The MCU BSP S/W is optimized with Green Hills MULTI Project Manager version v6.1.6 as Figure 5.2 MULTI Project Manager. The version of MULTI Project Manager can be checked via "Help→ About MULTI Project Manager" menu.

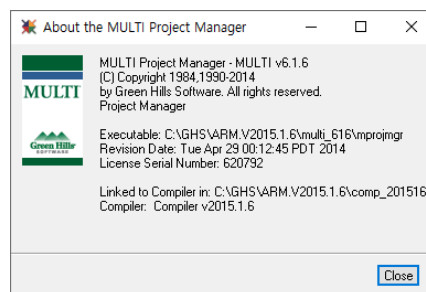


Figure 5.2 MULTI Project Manager

5.2.1.2 Green Hills Compiler Version

The following is the Green Hills compiler version of MCU BSP's. It is recommended to establish with the same version of compilers as Table 5.1 for maintaining compatibility of development environment with MCU BSP.

Table 5.1 Green Hills Compiler Version

<i>Type</i>	<i>Compiler Name</i>	<i>Version</i>
<i>C Compiler</i>	<i>ccarm.exe</i>	<i>Sun Oct 18 23:01:55 PDT 2015</i>
	<i>ccarm64.exe</i>	<i>Sun Oct 18 22:53:28 PDT 2015</i>
<i>C++ Compiler</i>	<i>comp_201516/cxarm.exe</i>	<i>Sun Oct 18 23:00:35 PDT 2015</i>
	<i>comp_201516/cxarm64.exe</i>	<i>Sun Oct 18 22:52:18 PDT 2015</i>
<i>Thumb Compiler</i>	<i>comp_201516/ccthumb.exe</i>	<i>Sun Oct 18 23:01:59 PDT 2015</i>
	<i>comp_201516/cxthumb.exe</i>	<i>Sun Oct 18 23:00:39 PDT 2015</i>
<i>Assembler</i>	<i>comp_201516/asarm.exe</i>	<i>Sun Oct 18 22:57:43 PDT 2015</i>
	<i>comp_201516/asarm64.exe</i>	<i>Sun Oct 18 22:46:27 PDT 2015</i>
<i>Linker</i>	<i>comp_201516/elxr.exe</i>	<i>Sun Oct 18 22:19:08 PDT 2015</i>
<i>Librarian</i>	<i>comp_201516/ax.exe</i>	<i>Sun Oct 18 22:09:51 PDT 2015</i>

5.2.2 Make New Project Using Green Hills IDE

This chapter describes the basic processor of own project setup using Green Hills IDE.

1. Write project name into **Name** field.

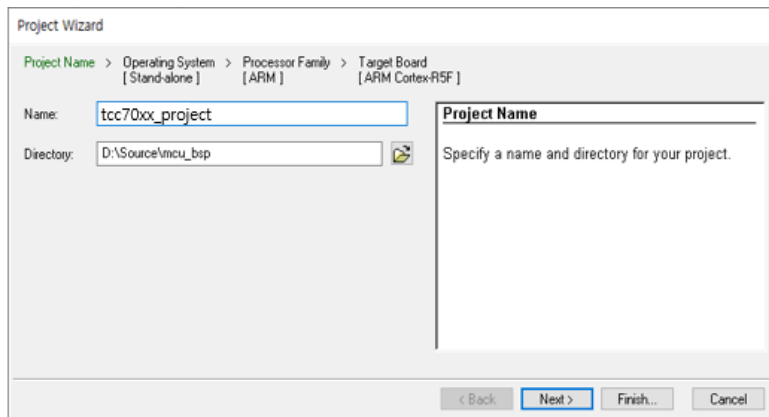


Figure 5.3 Write Project Name

2. Operating System: Select **Stand-alone**.

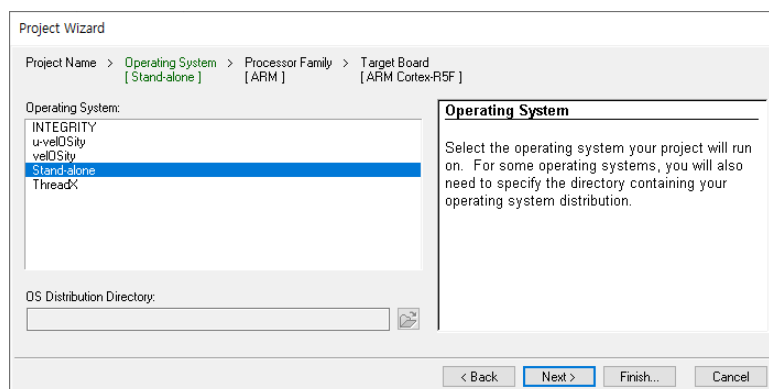


Figure 5.4 Select Operating System

3. Processor Family: Select **ARM**.

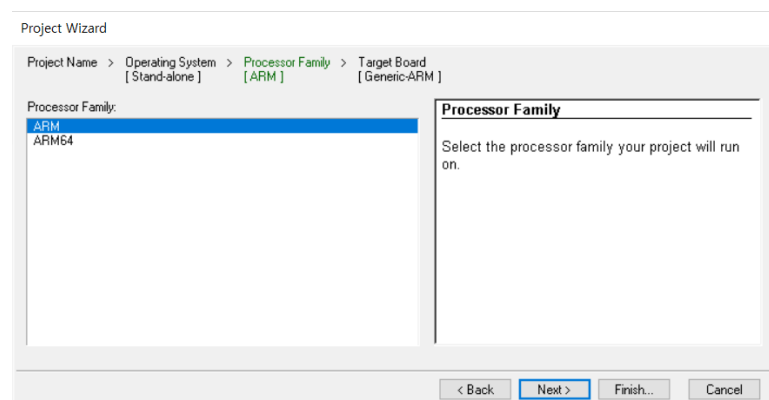


Figure 5.5 Select Processor Family

4. Target Board: Select **Generic-ARM:ARM Cortex-R5F**.

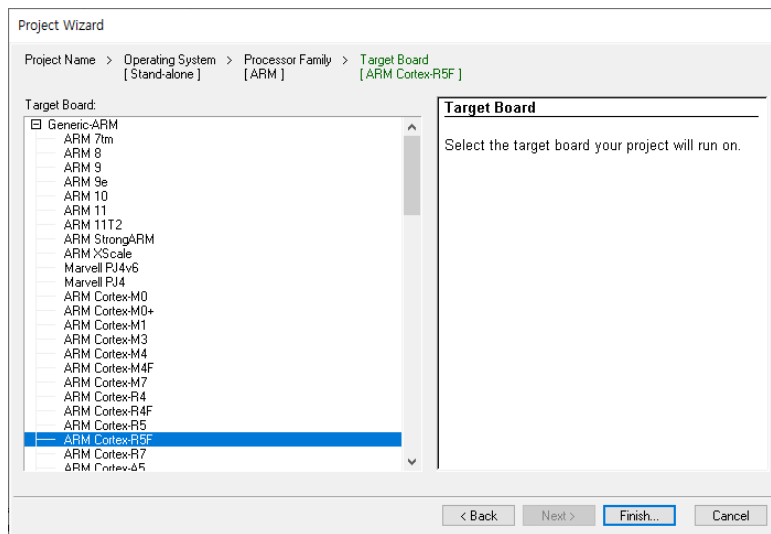


Figure 5.6 Select Target Board

5. Select **Finish** to activate, and then **Select Item to Add** Popup is shown.

- 1) In project directory, **.gpj** files and **tgt** folder are automatically generated as shown in Figure 5.7 Generated Project Files.

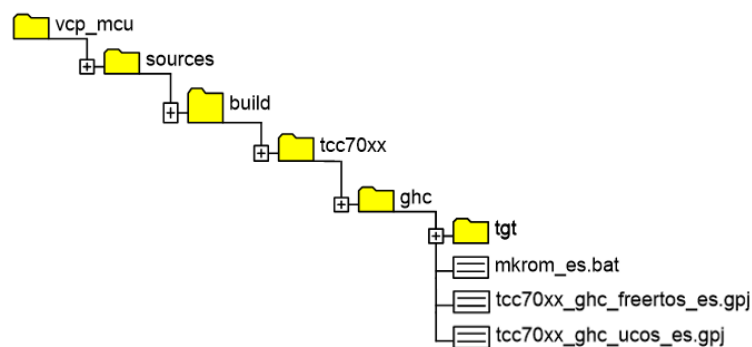


Figure 5.7 Generated Project Files

- 2) Whole setting of the project is managed in **Project_name.gpj** file. **Top Project** is the highest project file.
- 3) The project file consists of top project, sub project, program, and source code files.
 - Properties of the program are as follows:
 - Individual application source + basic elements of the LD (Linker Directives) file
 - If there is a header in the project folder, it is unnecessary to add it separately (Recognizing the dependency of the header)

6. Generate a new program.

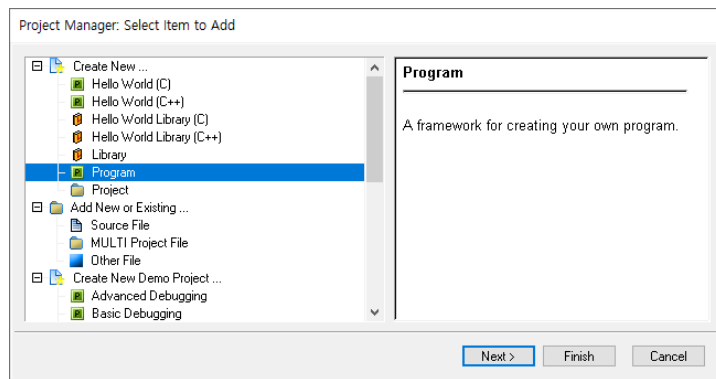


Figure 5.8 Generate New Program

1) Setting the source code directory and name: Select your source code directory and write project name as follows

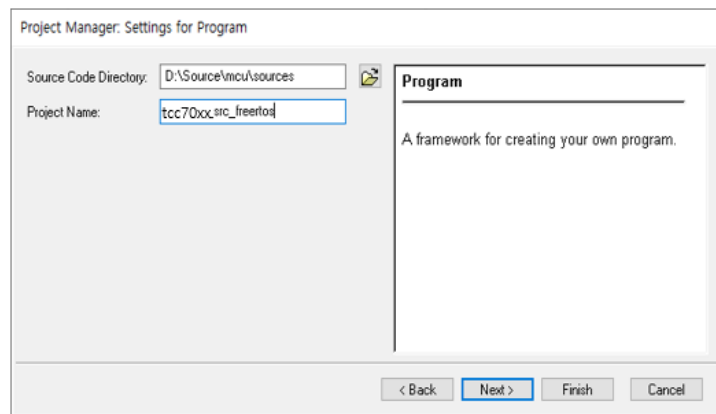


Figure 5.9 Select Source Code Directory

2) Setting the program layout: Set how to execute the program.

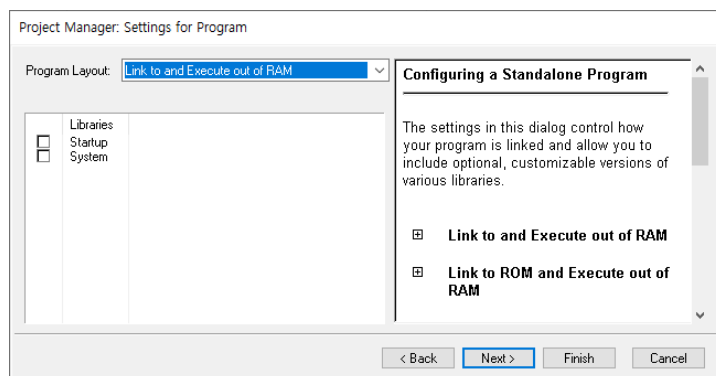


Figure 5.10 Select Program Layout

- Link to and execute out of RAM : Program code is loaded and executed in the RAM (Default setting).
- Link to ROM and execute out of RAM: Program code is loaded in the ROM and executed in the RAM. It is

available when the data area is copied to RAM in the startup code.

- Libraries are not selected (with own startup code).

3) Completion: Initial page is shown as follows.

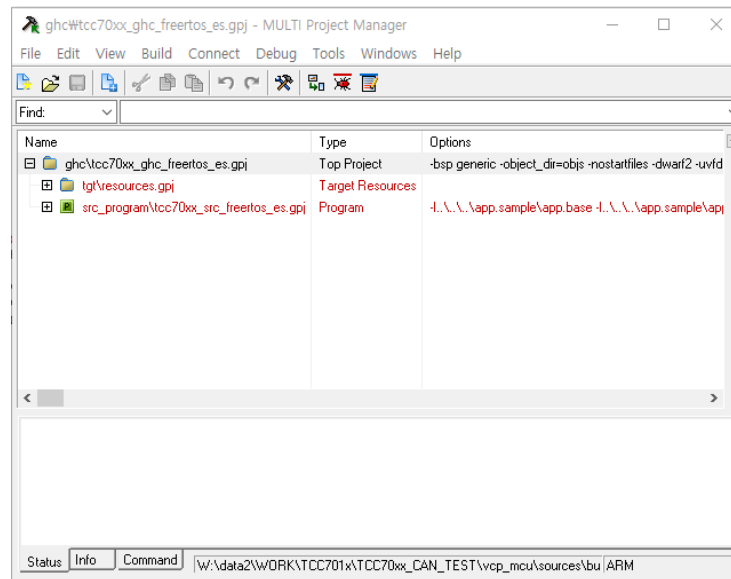


Figure 5.11 Completion of Initial Project Setup

7. Add Source Code.

- 1) Select program → Select , right-click, and select **Add Item** menu.

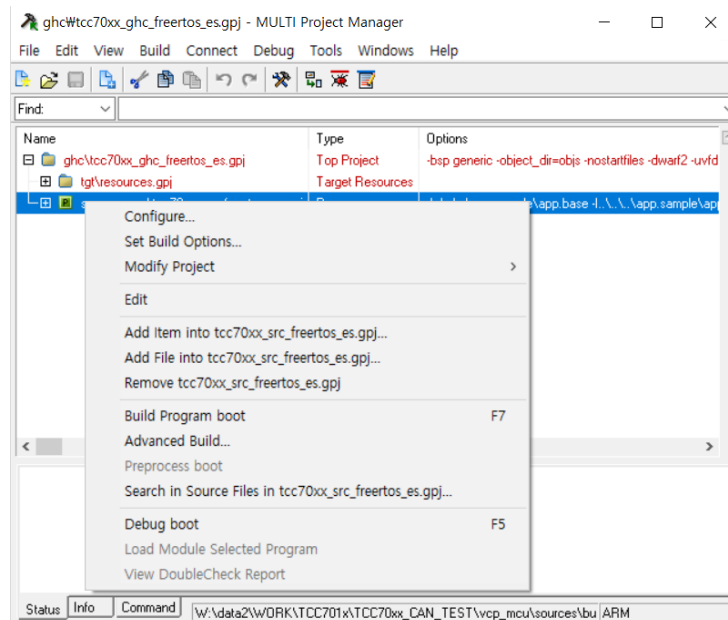


Figure 5.12 Add New Source Code Item

- 2) In **Select Item to Add** window → **Add New or Existing** → Add file in **Source File**.

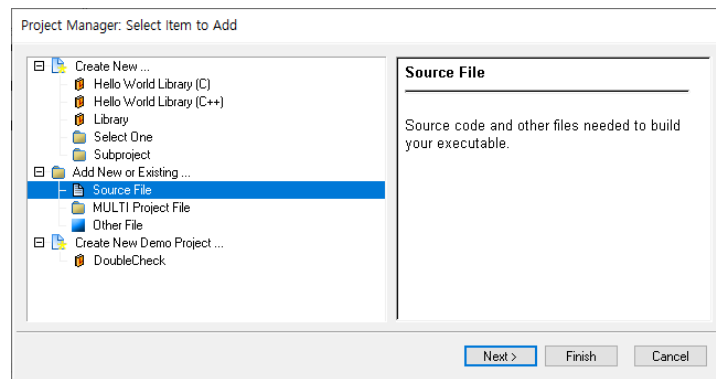


Figure 5.13 Add Source File

- 3) When you add the existing module individually or select the folder, .c/.h file is automatically added. You can also add your own modules in new project using drag and drop method.

- Search all of implementation file(.c) using wild character(*.c) on Windows browser.
- Select the files and drag and drop under program file (tcc70xx_src_freertos_xx.gpj).
- The result after adding is as follows.

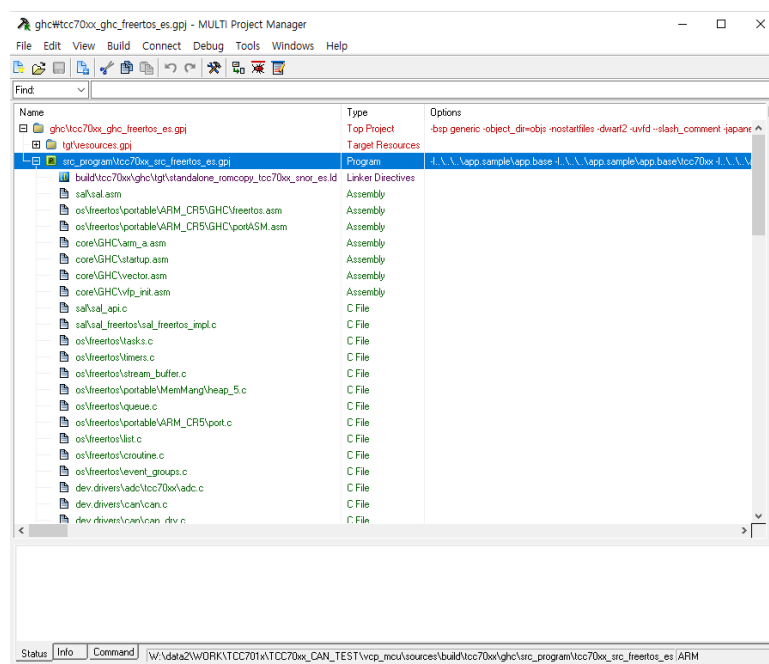


Figure 5.14 Complete Adding Source File

- 4) Add asm file: In **Select Item to Add** window → **Add New or Existing**: → Add files in **Other File**.

- Select the file type as **Assembly Source**.
- Add a file.
- The file extension distinguishes upper and lower case. Use lower case for .asm.

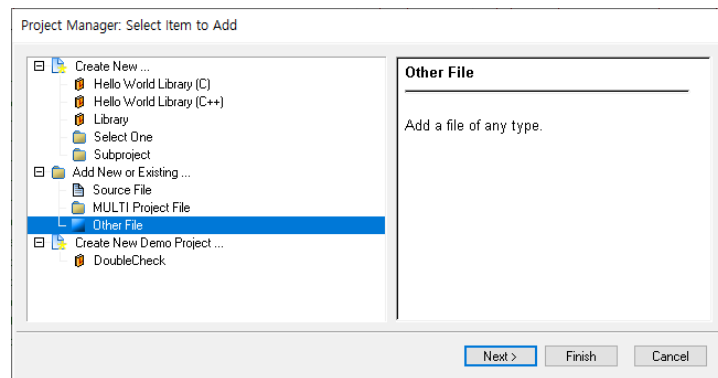


Figure 5.15 Add Assembly File (.asm)

5) How to add an include path

- Designate **include path** to add a header file as `#include <header.h>`
- Select a program → Right-click → Select **Set Build Options**.

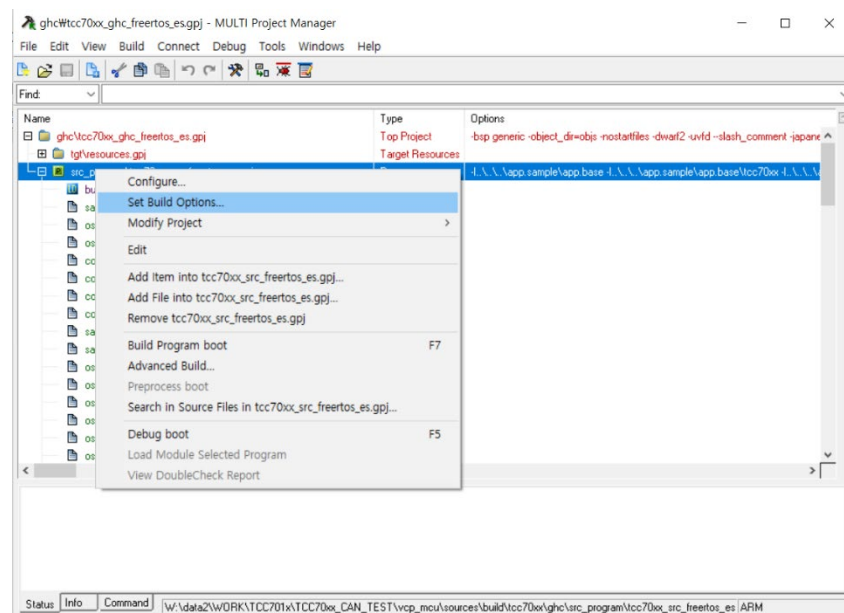


Figure 5.16 Set up Build Option

- Select **All Options** tab → Under **Project** → Select **Include Directories**.

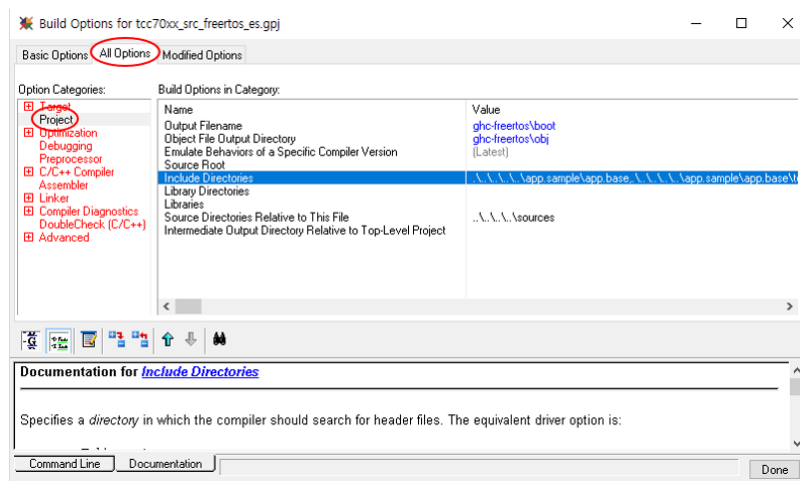


Figure 5.17 Configure Include Path

- Add new file path of a header file with relative path starting '\.' directive word. The Include directories use the relative path of the header file from the location of the program file (tcc70xx_src_freertos_xx.gpj).
 - If you find the tcc70xx_src_freertos_xx.gpj file under the "\ build\ tcc70xx\ ghc\ src_program\ " folder, see Figure 5.18 Add Include Path.

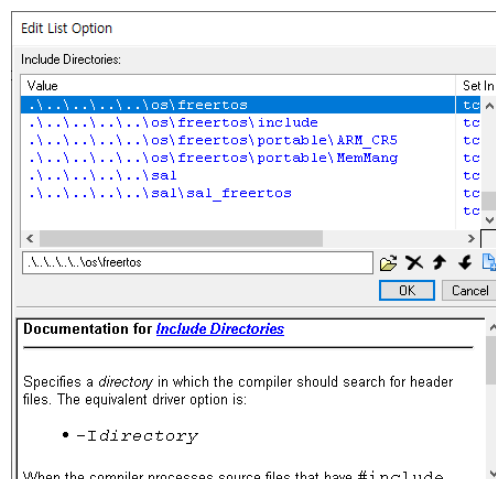


Figure 5.18 Add Include Path

- It is possible to add a path with **Program.gpj** file through the text editor.

```
#!/build
[Program]
-I../..../..../app.sample/app.base
-I../..../..../app.sample/app.base/tcc70xx
-I../..../..../app.sample/app.can.demo
-I../..../..../app.sample/app.console
-I../..../..../app.sample/app.key
-I../..../..../app.sample/app.fvug
-I../..../..../app.sample/app.spiled
-I../..../..../app.sample/test.app.audio
-I../..../..../app.sample/test.app.adc
-I../..../..../app.sample/test.app.audio
-I../..../..../app.sample/test.app.dse
-I../..../..../app.sample/test.app.eflash
-I../..../..../app.sample/test.app.fmu
-I../..../..../app.sample/test.app.gdma
-I../..../..../app.sample/test.app.gic
-I../..../..../app.sample/test.app.gpio
-I../..../..../app.sample/test.app.gpsb
-I../..../..../app.sample/test.app.hsm
-I../..../..../app.sample/test.app.i2c
-I../..../..../app.sample/test.app.ictc
-I../..../..../app.sample/test.app.lin
-I../..../..../app.sample/test.app.pdm
-I../..../..../app.sample/test.app.pmio
-I../..../..../app.sample/test.app.pmu
-I../..../..../app.sample/test.app.rtc
-I../..../..../app.sample/test.app.timer
-I../..../..../app.sample/test.app.vdt
-I../..../..../app.sample/test.app.sfmcc
-I../..../..../app.drivers/lin
-I../..../..../app.drivers/lin/tcc70xx
-I../..../..../app.drivers/hsm
-I../..../..../app.drivers/hsm/tcc70xx
-I../..../..../core
```

Figure 5.19 Add Include Path Manually

6) Memory Layout Setting

- Modify the basic .ld file. It is possible to select .ld file for each usage when compiling.
 - Select the program → Right-click → **Configure** → In **Setting for Program**, select the corresponding link option from **Program Layout**.

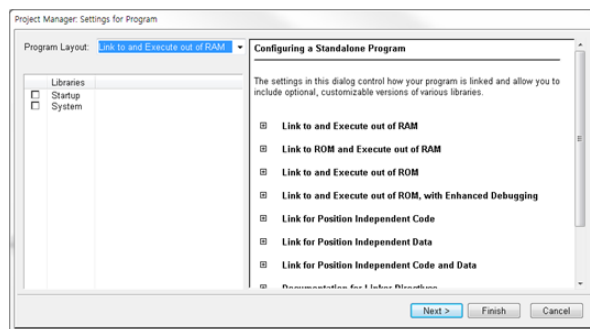
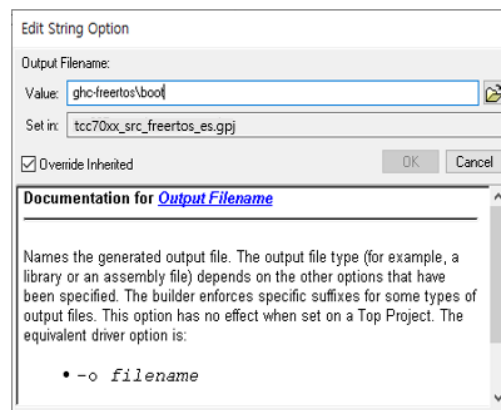


Figure 5.20 Set up Memory Layout

- It is possible to use .ld file written by you: Select **Custom** from **Program Layout** and the .ld file.
 - Using the pflash and Int.SRAM to copy **startup.asm .data, .bss** to the int.SRAM
 - standalone_romcopy_tcc70xx_snor_xx.ld

7) Output File Name Setting.

- When the 'Build' is normally completed, basically three files, "Program name".map (map file), "Program name" (no extension, elf file), and "Program name".mem (binary file), are generated.
- Changing **Output Filename**: Under **Program** → Right-click → Select **Set Build Options** → All Options tab → Project → Click **Output Filename**.

**Figure 5.21 Output File Name**

5.2.3 Build TCC70xx MCU BSP

1. The MCU BSP Software provides the optimized project files of freeRTOS and uC/OS-III RTOS.
2. Open project file (tcc70xx_ghc_freertos_xx.gpj) in "\\ build\\ tcc70xx\\ ghc" folder by using Green Hills MULTI IDE (In case of using uC/OS-III, use the tcc70xx_ghc_ucos_xx.gpj file). [xx can be es (evaluation sample) or cs (customer sample)]
 - 1) Double click **project file**.

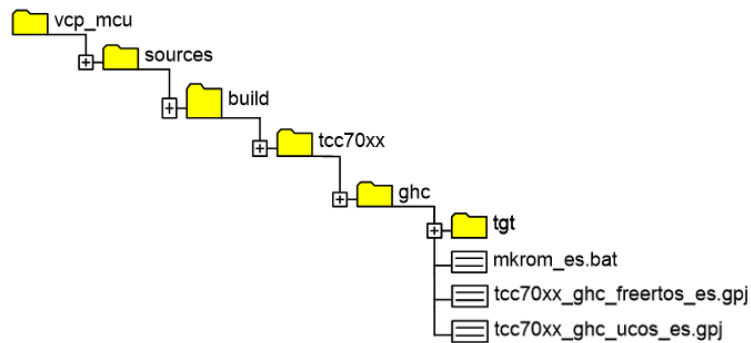


Figure 5.22 Open Project File

- 2) Modify definition in the source program file(.gpj) as below.

```

-D_func__=__FUNCTION__
-DDMA_NC_SIZE=0x4000
-DCAN_NC_SIZE=0x8000
-DSRAM_TOTAL_SIZE=0x40000
-DARM_STACK_SIZE=0x680
-object_dir=ghc-freertos\obj
-cpu=cortexr5f
-DPFLASH_STANDALONE
-DOS_FREERTOS
:postexecShellSafe='mkrom_es.bat freertos'
-O0
-DFMU_DRIVER
-DDEBUG_ENABLE=1
-DCONSOLE_HISTORY_ENABLE
-D__MPU_ENABLED__

```

Figure 5.23 Modify EVB Definition

- 3) Select build menu: **Build** → **Build program boot** or use **[F7]** shortcut key.

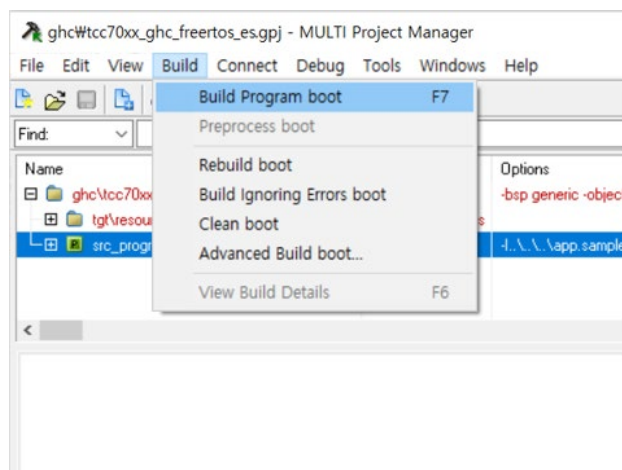


Figure 5.24 Build Project

- 4) Connect the executable binary image (boot.mem).

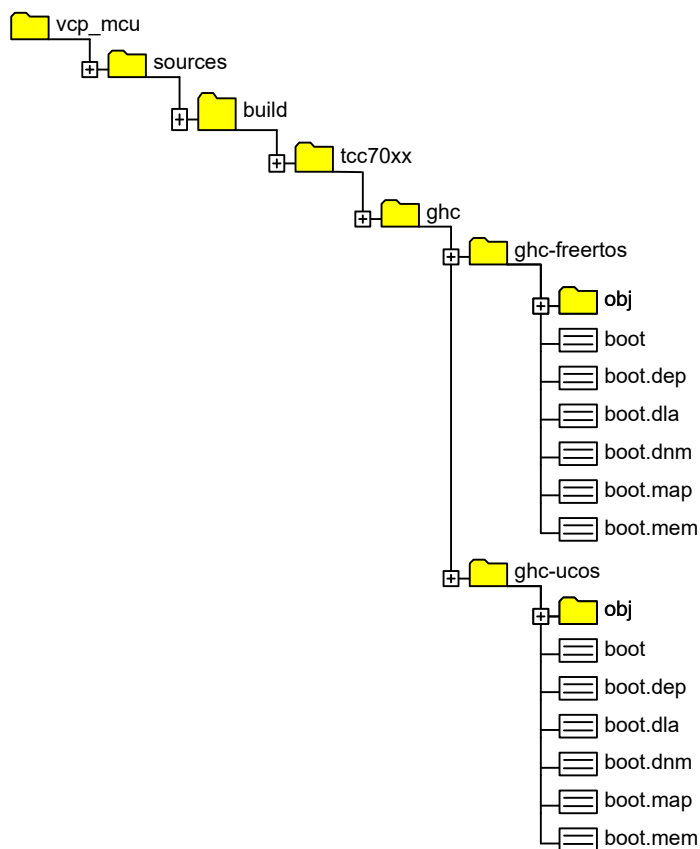


Figure 5.25 Build Completion

r5_fw.rom: Final MCU BSP rom file. It is created by adding a header to the binary file ("*boot.mem*"). It can be updated to the target EVB after being included in pflash image. Refer to Chapter 5.3 Preparation Image for Normal Booting.

5.3 Preparation Image for Normal Booting

To update MCU BSP rom file, you should create a pflash image. The pflash image contains some headers and F/W images related to normal boot and security. Basically, the pFlash image is automatically created after building the MCU BSP.

6 FIRMWARE DOWNLOADER (FWDN)

6.1 System Requirement

6.1.1 Software Requirement

Table 6.1 Software Requirement

Category	Description	Path	Note
VTC driver	vtcdrv v5.0.0.14.zip It is a driver for windows PC that is used to download firmware. VTC driver corresponding to the windows version must be installed. It is installed only once on your PC.	tools/fwdn_v8	You must use version 5.0.0.14 or higher.
FWDN	fwdn.exe Tool for downloading firmware.	tools/fwdn_v8	You must use version 8 or higher

6.1.2 Hardware Requirement

Table 6.2 Hardware Requirement

Category	Description	Note
EVB	Supported chipset are TCC70xx	
Power Adapter	12V adapter is used to supply power to the board and LCD.	Recommend using the power adapter (12V/10A) provided by Telechips.
USB C Type Cable	It is used to connect an EVB to PC for UART Log.	

6.2 Install VTC Driver

Install USB driver (VTC driver V5.0.0.14) for FWDN in the windows PC. The driver can be installed only once on PC. The VTC driver V5.0.0.14 is located in "*tools/fwdn_v8*".

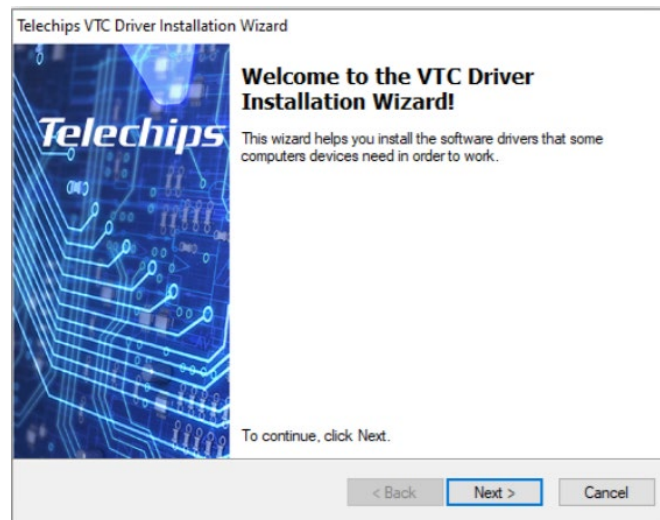


Figure 6.1 VTC Driver Install Window

6.3 FWDN Sequence

1. Change boot mode switch to USB boot mode and TCC7022_25_35_45_MCU EVB power off → on.
2. Move "*tools/fwdn_v8*" directory.
3. Execute FWDN and download pflash image. You can select executing FWDN command directly or using batch file.
 - 1) FWDN command
 - i. `fwdn.exe --write ..\..\sources\build\tcc70xx\gcc\output\tcc70xx_pflash_boot.rom -- fwdn .\ vcp_fwdn.rom`
 - 2) Sample batch file
 - i. `update_pflash_image.bat`: update pflash image
4. Input COM port number in the screen below.
You can check COM port number in the Device Manager of Windows system.

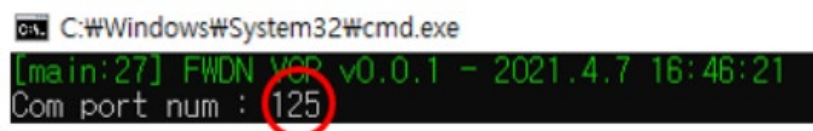


Figure 6.2 Input COM port number

5. Check the FWDN process
You can see the process of FWDN.
If FWDN process is completed, "Complete FWDN" is shown.

```

C:\Windows\System32\cmd.exe
Com port num : 125
[ProtocolCB::OpenPort:35] Complete open port(###.WCOM125)
[ProtocolCB::StartVCPFWDN:45] Complete to receive start res
[FWDN_VCP::LoadFwdnFW:106] Complete to send start msg
[ProtocolCB::CheckResPacket:166] Complete to receive ack for cmd(0xFFFF0000(RECEIVE_HSM_CMD))
[FWDN_VCP::WriteFile:239] Complete to send command(0xFFFF0000(RECEIVE_HSM_CMD))
[ProtocolCB::SendFile:145] Complete to send file
[ProtocolCB::CheckResPacket:166] Complete to receive ack for cmd(0xFFFF0000(RECEIVE_HSM_CMD))
[FWDN_VCP::WriteFile:250] Complete to send file - .\tcc70xx_pflash_boot.rom
[FWDN_VCP::LoadFwdnFW:125] Complete to send hsm
[ProtocolCB::CheckResPacket:166] Complete to receive ack for cmd(0xFFFF0001(RECEIVE_FWDN_CMD))
[FWDN_VCP::WriteFile:239] Complete to send command(0xFFFF0001(RECEIVE_FWDN_CMD))
[ProtocolCB::SendFile:145] Complete to send file
[ProtocolCB::CheckResPacket:166] Complete to receive ack for cmd(0xFFFF0001(RECEIVE_FWDN_CMD))
[FWDN_VCP::WriteFile:250] Complete to send file - .\wcp_fwdn.rom
[FWDN_VCP::LoadFwdnFW:135] Complete to send fwdn
[FWDN_VCP::LoadFwdnFW:141] Complete to load FWDN F/W
MT
MR0=0000a042
MR1=00020018
MR2=00000000
RM=00000000
[ProtocolFW::CheckResPacket:259] Complete to receive ack for cmd(0xAAAA0014(STORAGE_INFO_CMD))
[FWDN_VCP::InfoStorage:55]
#### SNOR Info ####
Manufacture ID: 0xc2
Device ID: 0x2017
Name: MXIC-MX25L6435E
Sector Size: 4 KiB (4096 Byte)
Total Capacity: 8 MiB (8388608 Byte)
4Byte Address Mode: Unsupported
#### EFLASH Info ####
DCYCRDICON 0x1e0002
DCYWRCON 0x20100
Sector Size: 8 KiB
Page Size: 2 KiB
----Storage init info----
0 : Init success
X : Init failed or not exist
SNOR : 0
eFlash : 0

[ProtocolFW::CheckResPacket:259] Complete to receive ack for cmd(0xAAAA0017(CHIP_INFO_CMD))
[FWDN_VCP::PrintBankInfo:421] ---bank info---
bank = 0
eFlash offset : 0x0
eFlash size : 2097152 byte
SNOR offset : 0x0
SNOR size : 0 byte
[FWDN_VCP::PrintStorageOption:404] ---storage info---
eflash
offset : 0x0
size : 733568 byte
[ProtocolFW::CheckResPacket:259] Complete to receive ack for cmd(0xAAAA0011(WRITE_CMD))
[FWDN_VCP::WriteFile:239] Complete to send command(0xAAAA0011(WRITE_CMD))
[ProtocolFW::SendFile:131] Complete to send file
[ProtocolFW::CheckResPacket:259] Complete to receive ack for cmd(0xAAAA0011(WRITE_CMD))
[ProtocolFW::CheckResPacket:270] Received CRC : fdfdfcfc, Calculated CRC : fdfdfcfc
[FWDN_VCP::WriteFile:250] Complete to send file - .\tcc70xx_pflash_boot.rom
[main:121] Complete FWDN

```

Figure 6.3 FWDN Process

- Change boot mode switch to normal boot mode and TCC7022_25_35_45_MCU EVB power off → on.
- Check whether MCU BSP is running via UART log. If you want to see log below, change "MCU_BSP_BUILD_FLAGS_DEBUG" flag to 1 from 0 in Makefile.

```

Initialize System done
Welcome to Telechips MCU BSP

MCU BSP Version: V0.1.0
(D)[SAL 1](FR_OSStart:481) ~ Done to initialize Free RT Operating System ~

```

Figure 6.4 Initial Log of MCU F/W

Note: The files for FWDN are included in prebuilt images such as "hsm.bin".

7 HOW TO DEBUG BY USING JTAG DEBUGGER

You can debug software by using JTAG debugger device such as Trace32, CodeViser and so on. The script for Trace32 is supported but it is not official. For more detailed information on Trace32 script, contact Trace32 supplier. TCC70xx MCU BSP was tested under the following conditions.

- Hardware : TCC7022_25_35_45_MCU EVB V0.2.1
- Software : TCC70xx MCU BSP V0.1.0
- Compiler : GCC
- JTAG debugger : Trace32

The sequence below shows debug method using Trace32 scripts.

1. Connect JTAG probe of JTAG debugger device and JTAG connector of TCC7022_25_35_45_MCU EVB.
2. Change boot switch to USB mode.
3. Download "tcc70xx_pflash_boot.rom" via FWDN.
4. Change boot switch to normal boot mode.
5. Reboot TCC7022_25_35_45_MCU EVB.
6. Run Trace32 PowerView (Trace32 PC program).
7. Open script (Trace32 PowerView → File → Edit Script → select "*vcp_mcu/scripts/debug/T32/ vcp_pflash_attach.cmm*")
8. Edit image path and save.
9. Click **Do** button.

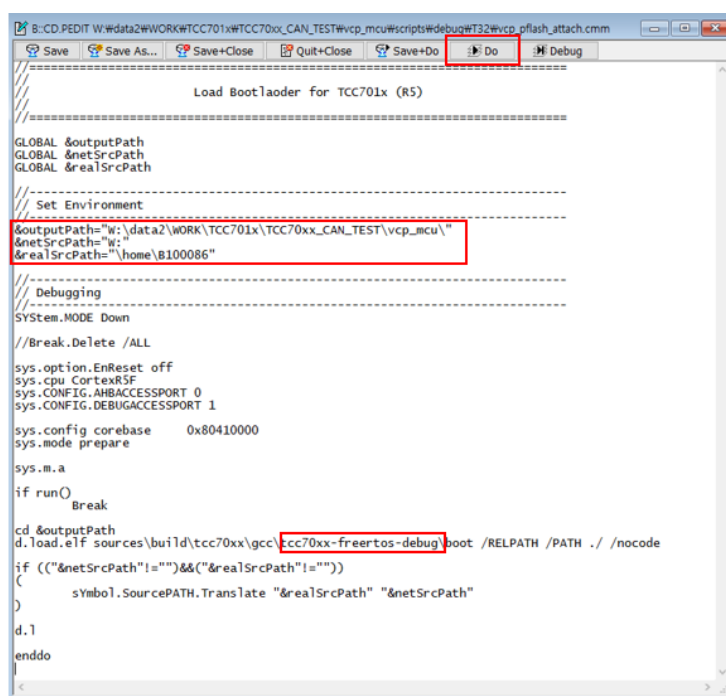


Figure 7.1 Trace32 Script for Debug

You can debug software if the sequence above is normal. For details on Trace32, see the manual of Trace32.

8 REFERENCES

- [1] TCC70xx Full Specification
- [2] TCC70xx Common Hardware-Quick Start Guide for TCC7022_25_35_45_MCU EVB
- [3] TCC70xx Common Hardware-User Guide for TCC7022_25_35_45_MCU EVB
- [4] Contact Telechips for more details: sales@telechips.com

9 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."